

Cyber Security: threats, attacks & the human factor

Frameworks set the standard, vulnerabilities open the door, and attackers walk through either the person or the code. Every risk on this side has a defence on side 2.

01 Standards Essential Eight, ISM, ethics — what "secure" is measured against.	02 Vulnerabilities Weaknesses in insecure dev environments, and the breaches they caused.	03 Attacks MitM on the wire, social engineering on the person, injection in the code.
--	---	---

01 Industry Frameworks & Ethics

W1 L2-L3

Essential Eight — ACSC baseline, maturity levels 0-3 1 Application control — blocks unapproved software. 2 Patch applications — closes known flaws. 3 Configure Office macros — macros carry malware. 4 User app hardening — disable Flash, ads, Java. 5 Restrict admin privileges — trusted users only. 6 Patch operating systems — keep the OS current. 7 Multi-factor auth — beyond passwords. 8 Regular backups — restore after ransomware.	ISM — the Australian Government's primary framework G1 Environments Separate dev / test / production · limit access per environment · never use un-anonymised production data · log all access. G2 Secure design Secure coding standards (input validation, output encoding) · threat modelling · peer code reviews · regular training. G3 Security testing SAST static (code not running) · DAST dynamic (running app) · penetration testing · remediate then retest. Framework = a structured set of best practices, standards and guidelines keeping technology and data secure, compliant, efficient and maintainable.	Ethics & the five issues Ethics — accepted rules of behaviour. Code of ethics — principles for acting honestly and with integrity; enforced by law in banking/finance, voluntary elsewhere. ACS Code — honesty, trustworthiness, respect (also ACM, IEEE, AI Ethics Principles). Ineffective security — weak encryption or validation exposes users. Use of AI — question training data, accuracy and bias. IP & copyright — respect licences, credit creators, check library terms. Privacy & metadata — timestamps, location and device info identify people. Content moderation — duty to filter and flag offensive or unlawful content. Ignoring ethics → lost trust, reputational damage, legal action or bans, lasting harm.
--	---	---

02 Vulnerabilities, Risks & Australian Case Studies

W2 L1 — HIGHEST-YIELD

VULNERABILITY	CASE STUDY	WHAT HAPPENED → KEY MITIGATION
Poorly secured APIs	Optus, 2022	Exposed API, weak access control, 9M+ Australians → strong auth on every endpoint, HTTPS, limit data returned, rate limiting, audits.
Malware	Toll Group, 2020	Two ransomware hits in 3 months via phishing + outdated software → anti-malware, scan dependencies, whitelisting, training.
Unpatched software	Accellion, 2021	Unpatched file-transfer flaw exfiltrated government documents → automated patch management, vendor advisories, staging tests.
Weak IAM	ANU, 2023	Compromised credentials, no MFA , poor logging → least privilege, MFA on privileged accounts, unique (never shared) credentials, review logs.
Insider threats	Fintech startup, 2022	Departing employee planted backdoors → version control + code review, revoke access on exit, monitor logs, ethics training.
MitM attacks	Banking apps, public Wi-Fi	Traffic redirected to fake logins → HTTPS, VPN on public Wi-Fi, certificate pinning, network segmentation.
Security incidents	Service NSW, 2021	Phishing took staff email, 700,000+ documents exposed → incident response plan, IDS/IPS + logging, tested backups.
Third-party software	SolarWinds, 2020	Backdoor in a legitimate update reached federal departments → trusted sources only, audit third-party tools, track vulnerability disclosures.
Poor code review	Retailer checkout	Logic flaw missed in an informal review cost promotional sales → structured peer review, linters / static analysis, checklists.
Combined dev/test/prod	Health provider, 2019	Live patient data on a test server indexed by search engines → isolate environments, never use real customer data, secure all tiers equally.

Framing — insecure development environments aren't properly protected, letting attackers reach data, disrupt services or destroy trust. **IoT Code of Practice** — 13 principles; prioritise no default/weak passwords · a vulnerability disclosure policy · secure updates.

03 Man-in-the-Middle

W2 L1

How it works An attacker secretly intercepts or alters communication between two parties who think they're talking directly — separate connections with each side, messages relayed, so it looks normal while monitored or controlled. Easiest on unencrypted Wi-Fi and email. Encryption helps, but users are still lured to phishing sites and traffic recorded for later. Attackers want to spy on data, steal credentials, redirect transfers and corrupt data.	Seven techniques SSL stripping — HTTPS to the site, HTTP to you. Evil twin — fake hotspot posing as trusted. Email hijacking — impersonate mid-thread, redirect funds. Wi-Fi eavesdropping — trustworthy-looking free networks. Session hijacking — steal cookies / session keys. DNS spoofing — altered records send users to a fake site. IP spoofing — disguise the source as trusted.	Spoofing & invoice scams Email spoofing — forged sender; users see only the display name, so they click, open or pay. Caller ID spoofing (CLI oversteamping) — fake local number; legal in Australia unless used maliciously. Invoice scamming — a real invoice intercepted, bank details swapped, money moved offshore. The payer usually wears the loss. 1 in 5 Australian small businesses in 2021, avg. \$15,500. Prevent — email and web security tools · staff education with real cases · strong unique passwords changed every 3 months.
--	---	--

04 Social Engineering & Mitigation Measures

W2 L2-L3

Six attack types Psychological manipulation fooling users into handing over confidential data, usually invoking urgency or fear. Hard to prevent — the weak point is human. Phishing Most common — harvests names, passwords, PINs and account numbers via misleading links and urgency. Spear phishing Targeted — tailored to a person's job, contacts and characteristics. Baiting Something enticing — a USB labelled "Confidential" that auto-installs malware. Quid pro quo A "service" for access — fake IT help, or \$100 for network access. Pretexting False trust by impersonation — "IT support" or an "auditor". Tailgating Physically following staff into a restricted area, or borrowing a device.	Mitigation — ACSC & OAIC Credentials are stolen via phishing sites · brute-force guessing · breached services · malware capturing logins. 1 Staff awareness — train to spot phishing and spear phishing. Organisationally: employee education (most effective) · verification policies · high spam filtering · secure, updated devices. 2 Password safety — long unpredictable passphrases · no reuse · change regularly · lock after failed attempts · reset compromised passwords at once. 3 Secure software practice — MFA for remote access and important actions · patch software, browsers, plugins · anti-virus · monitor accounts · never enter credentials via emailed links.	Proactive vs reactive training Security awareness — proactive. Ready <i>before</i> the attack: spot scams, know the risks, report problems, refresh regularly. Builds safer habits. Incident response — reactive. <i>After</i> it goes wrong: notice it, contain the damage, preserve evidence, notify IT, remediate. Builds fast reactions. Seatbelts and airbags — awareness stops things going wrong; incident response minimises damage when they do. Together they form a strong defence.
---	--	---

05 Attacks on the Code: XSS & SQL Injection

W2 L4

Security by Design & OWASP Top 10 SbD — security is in the plan from the very beginning, not bolted on: build the house with the locks already in. Protect data before the first line of code. 1 Broken access control 2 Cryptographic failures 3 Injection 4 Insecure design 5 Security misconfiguration 6 Outdated components 7 Auth failures 8 Integrity failures 9 Logging failures 10 SSRF	Cross-site scripting (XSS) A flaw letting an attacker inject code (usually JavaScript) into a web page other users will see, which their browsers run unknowingly — caused by displaying user input without checking or cleaning it. Impact: change the site's look or behaviour · steal cookies · act as the user · spread between users like a virus. Stored XSS — the code is saved in the database and re-served, e.g. a username <code>user123<script>...document.cookie.</script></code> sending every visitor's cookies to the attacker. Acts like a worm, spreading profile to profile. Prevent (zero trust) — sanitise input by escaping special characters · validate type, length, range and format server-side · encode output · allow users to disable client-side scripts. Client-side validation alone is never enough.	SQL injection Malicious code inserted into a normal SQL statement to dump a database, produced through unsanitised user input. Easily prevented by validating input — but often isn't. Incorrectly filtered escape characters — input passed straight in: <code>"SELECT * FROM users WHERE username='"+username+"'"</code> The attacker adds an always-true clause, comments out the rest, or uses <code>;</code> to run a second statement. Incorrect type handling — no type check, so the string ends early and a command is appended. Prevent — assume all submitted data is unsafe and validate it (e.g. <code>mysql_real_escape_string()</code>) · parameterised statements (placeholders also fix type handling) · prepared statements & stored procedures (not a full fix) · least-privilege DB permissions · WAF / database firewalls (slow attacks, don't stop them).
--	---	--

Defences: law, data, controls & evaluation

The legislation that binds development, how data is protected and destroyed, the controls and practices that keep code safe, build-vs-buy, threat modelling, and the criteria used to judge whether any of it works.

04 Comply Three Acts, APPs and IPPs, secure data, backups, disposal.	05 Control Six controls, onboarding, training, risk plans, in-house vs external.	06 Evaluate Threat modelling, then the eight criteria for judging security.
--	--	---

06 Key Legislation

W2 L5 – COLLECTION, STORAGE & COMMUNICATION OF DATA

- **Privacy Act 1988** — federal

Covers: Commonwealth departments · ACT agencies · private organisations with turnover ≥ **\$3M/yr**, holding health information, trading personal information, or contracted to the Commonwealth.

- Collect only what is necessary; use only for the intended purpose; no third-party disclosure for other purposes.
- Individuals may access and correct their information.
- A public privacy policy is required, and steps taken to ensure data quality and security.

APPs — 1 open & transparent management · 3 collection of solicited info · 6 use or disclosure (purpose-bound unless consented) · 8 cross-border disclosure · 9 government identifiers · **11 security** (encryption, access controls, testing).

- **Privacy & Data Protection Act 2014** — Victoria only

Covers: Victorian departments and agencies · Victoria Police · state schools · public universities · public hospitals (**non-medical only** — medical is the **Health Records Act 2001**) · government-funded service providers.

Agencies safeguard data with security assessments and encryption. Breach notification is **encouraged, not mandatory**. The **Victorian Information Commissioner** oversees compliance; penalties include legal action or administrative orders.

IPPes — 1 collection (fair, lawful, not intrusive) · 2 use & disclosure · **4 data security** (4.2 destroy or de-identify data no longer needed) · 5 openness · 7 unique identifiers · 9 transborder data flows · 10 sensitive information.

- **Copyright Act 1968** — federal

Protects **intellectual property** — original creations of the mind: text, video, software, artistic works. Updated by the **Copyright Amendment (Digital Agenda) Bill 2000**.

- **Ownership** — the creator's unless assigned; **employers** hold copyright over work produced as part of a job. **Automatic and free** once in tangible form.
- **Protected:** literary, dramatic, musical and artistic works, films, recordings, broadcasts. **Not:** ideas, concepts, names, slogans, images of people.
- **Personal use exception** — format-shifting media you legally own.

Infringement — civil penalties (cessation orders, damages) and, in severe cases, criminal charges and imprisonment.

07 Security & Disposal of Data

W3 L1

- **Secure data**

Data security — only those who should **read** data can read it, and only those who should **write** it can write it (user authentication, password policies).

Secure information is: protected — authentication, firewalls, current anti-malware, physical locks, guards, alarms · **confidential** — need-to-know access only, required by the **PDP Act 2014** · **risk managed** — regular backups, ventilation and fire safety in data centres, a business continuity plan.

Why data matters — hardware is insured and replaceable; the data isn't. A business loses client lists, payroll, tax and accounting records and debts owed — staff go unpaid, income stops, creditors and the ATO act, trade secrets and reputation are gone.

- **Backups & the 3-2-1 rule**

A **backup** is a copy recoverable after a primary failure — hardware/software failure, corruption, malware or accidental deletion. The longer between backups, the more is lost.

3 **Three copies** — the original plus two backups.

2 **Two media types** — different kinds of storage.

1 **One offsite** — away from the original location.

3-2-1-1 keeps two offsite copies: one cloud, one offsite *and* offline. Daily incremental to portable/cloud, end-of-week server, quarterly and yearly — and **test that you can restore**.

E8 maturity — ML1 monthly · ML2 weekly, offline or non-rewritable · **ML3 (aim here)** at least daily, offline or non-rewritable, kept 3+ months, full restore tested at implementation and on every infrastructure change.

- **Disposal & destruction**

Data disposal — destruction, transfer, damage or alteration of data. **IPP 4.2:** destroy or de-identify information no longer needed. **IPP 1.1:** collect less, so there's less to dispose of.

Public Records Act 1973 — no destruction without **PROV** authority: **NAP** (normal administrative practice — transitory messages, rough papers, drafts, reference copies), **RDA, SIDA**.

Deletion ≠ destruction. Deleting removes only the **pointer** (name and path); the data survives until overwritten and can be recovered.

Sanitisation — **clearing** (overwritten so recovery tools fail) · **purging** (randomised against a laboratory attack) · **degaussing** · **destruction**. **Physical** — shredding (cross-shred if sensitive), pulping, incineration, pulverisation. Destroy **all** copies — backups, cloud, offsite. Factory reset isn't enough: **encrypt before resetting** phones, tablets and SSDs.

08 Security Controls & Improving Development Practice

W2 L6 • W3 L2

- **Six essential controls**

Version control	Tracks and manages code — controls access, recovers from errors.
IAM	Manages user access — prevents unauthorised access.
Encryption	Protects data — safe even if breached.
Code review	Peer-checks code — finds security issues early.
Updates / patches	Fixes vulnerabilities — reduces known exploits.
Environment separation	Isolates stages — keeps real data and users safe.

- **Version control & IAM**

Version control (Git / GitHub) tracks every change across the project lifecycle — without it each change overwrites the last. Repositories record **who changed what, when and why**, restrict who can merge, and let malicious code be traced and removed. A collaboration tool *and* a **safety net** — but still back it up.

IAM — each person gets only the access they need, via credentials, MFA and role-based permissions, so a stolen password does limited damage.

Least privilege — a tester views results but can't change source; a junior developer builds a feature without touching the production database.

- **Improving development security**

Secure development — safe by design at every stage; protects user data, prevents costly post-release fixes, builds trust, meets legal standards.

1 **Onboarding** — day-one briefing · least-privilege access · codebase walkthrough of encryption, auth and error handling · name the security contacts.

2 **Developer training** — common vulnerabilities (XSS, SQLi, broken auth) · safe coding standards · security tools · OWASP Top 10, via courses, workshops, refreshers and code reviews.

3 **Risk management plans** — **identify** → **assess** (likelihood × impact) → **plan responses** → **monitor**. Risks: insecure libraries, missing validation, poor authentication, delayed patching. **Register:** risk · cause → likelihood · impact · level → strategy · action · owner → monitoring · status.

09 In-House vs External Development & Third-Party Risk

W4 L1

- **Advantages & disadvantages**

In-house — full control · tailored to goals · efficient stakeholder communication · adapts quickly.

In-house — resource intensive (devs, PMs, testers, support) · higher cost · slow for complex features.

External — cuts time and cost · frees the team to innovate rather than rebuild basics.

External — trusts another's security and update cycles · vendor abandonment or library flaws disrupt you · licensing risk.

Most teams combine both — in-house stays popular where data is sensitive or requirements unique.

- **Factor-by-factor**

FACTOR	IN-HOUSE → EXTERNAL
Control	Full → limited over external code and customisation.
Cost	High (staff, tools, infrastructure) → cheaper via reuse.
Speed	Slower → faster with reusable components; maintenance shifts to contractors.
Security	Easy to audit, data stays in-house → relies on vendor practice.
Legal	Owns the IP → must comply with licences and vendor terms.

- **Third-party risk & SCA**

Third-party software — any external SDK, API, plug-in or hosted app. Saves time and cost, but badly managed it becomes the **weakest link**.

Risks: library vulnerabilities exploitable even when your code is secure · excessive access to files and data · flaws left unpatched · vendors abandoning products.

Software Composition Analysis scans the codebase to identify every external component, flag known vulnerabilities, check licence compatibility, recommend updates, and show whether vulnerable code is actually called.

Controls — encrypt data in transit and at rest · grant the **minimum access** needed · monitor and log all external access · review and revoke permissions regularly.

10 Threat Modelling

W4 L2 – THINK LIKE AN ATTACKER

PRINCIPLE 1
Define security requirements

What protections does the system need? e.g. **"passwords must be encrypted"**, "only verified users access private data". "Log all failed logins". Align with law (**Privacy Act 1988**), support authentication and authorisation, and **document from the start** — late security is harder and dearer to fix.

PRINCIPLE 2
Identify threats

"What could go wrong?" **Data breach** (leaked database) · **code injection** (SQLi via a login form) · **XSS** (script steals cookies) · **DoS** (spam logins crash the server) · **Mitm** (eavesdropping on open Wi-Fi). **STRIDE** classifies them: Spoofing, Tampering, Repudiation, Information disclosure, Denial of service, Elevation of privilege.

PRINCIPLE 3
Mitigate threats

SQLi → input validation + prepared statements. Data breach → encrypt stored data, secure authentication. XSS → sanitise input. DoS → rate limiting and traffic filtering. Weak passwords → strong policies + **MFA**. Plus penetration testing, secure coding training and **SAST / DAST / IAST** tools.

PRINCIPLE 4
Confirm mitigation

Verify the threats are genuinely handled: **security testing** (manual and automated) · **code reviews** · **regression testing** so fixes don't break other parts · **audit trails** · **compliance testing** against legal and policy standards. Security is **continuous**, not a one-time fix.

11 Evaluating the Security of Development Practices

W4 L3 – CRITERIA FOR THE EXTENDED RESPONSE

CRITERION	ASK	CRITERION	ASK
Risk evaluation	Are all threats identified, judged on likelihood and consequence , and mitigations reviewed for effectiveness?	Third-party	Are all external tools documented and tracked , reviewed for known vulnerabilities (update history, patches, support, via SCA), and licence-checked first?
Security controls	Are authentication and access control configured properly, data encrypted in storage and transmission , controls monitored and updated? Outdated controls or default passwords are as bad as none — over-strict ones block legitimate users.	Awareness & culture	Does everyone — developers, testers, managers, users — get regular training, with security a shared responsibility prioritised at every level ?
Dev practices	Are developers trained in secure coding, input validation and error handling applied consistently, and least privilege and fail-safe defaults in the code?	Incident readiness	Is there a documented and tested response plan — who is notified, how systems are isolated, data recovered, communication handled? Are roles defined and incidents reviewed afterwards?
Security testing	Are scans and penetration tests run regularly, especially after changes? Are security tests in the CI/CD pipeline ? Do results feed back into code and architecture?	Compliance	Do practices comply with the Privacy Act 1988 , PDP Act 2014 and ACSC guidance, and are strategies improved using audit and incident feedback? Compliance is the floor, not the ceiling.

Answer shape — name the criterion —> state the evidence from the case study —> judge it against the standard (framework, Act or best practice) —> recommend the control that closes the gap, and say how you'd confirm it worked. The goal is never zero risk — it is risk at an **acceptable level**.